APPLICATION SECURITY LAB / VERIFIED TEST RESULTS

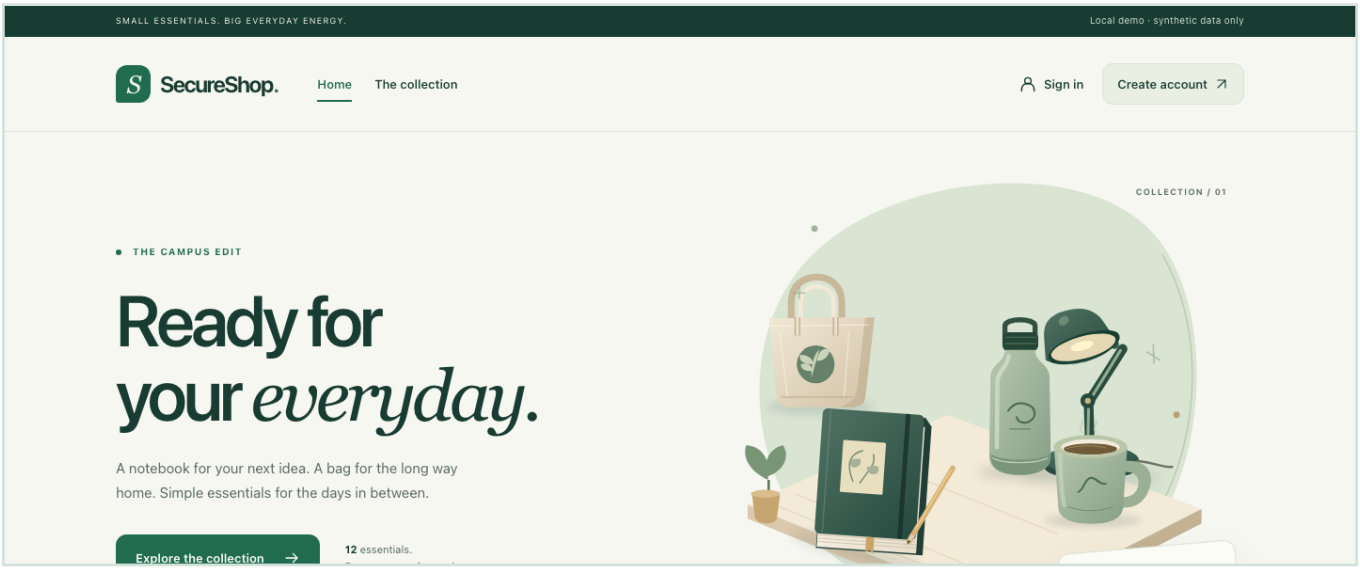
SecureShop

Functional testing & ZAP baseline

08 October 2026 | Asia/Kolkata | Local target: http://127.0.0.1:8080

Assessment complete. Vulnerable baseline retained. Customer and administrator journeys work in the tested environment. All four controlled weaknesses remain present; remediation was explicitly excluded.

34	3	4	10
Regression checks passed	Fresh-seed demo runs passed	Controlled weaknesses confirmed	ZAP alert types reviewed



Verified desktop home page - top viewport of the original browser capture.

The scan produced **270 alert instances**: 1 high, 97 medium, 131 low and 41 informational. Repeated URLs and scan phases create multiple instances; these are not 270 independent vulnerabilities.

Review items: one moderate accessibility naming issue; 11 incomplete axe contrast results; nine low-confidence potential-XSS hotspots with no demonstrated XSS. Source, data, original evidence and the vulnerable tag were preserved.

Scope, setup & provenance

The user authorized a complete local test suite, a ZAP baseline scan, and PDF plus Markdown documentation. No fixes, remediation tag, external target scan, payments or real customer data were authorized or performed.

Browser	ZAP 2.17.0	Flask	SQLite
Normal UI journeys	127.0.0.1:8090	127.0.0.1:8080	Isolated shop.db

Item	Observed value
Tested application source	a564617796a857dd482e8fe6daa7244d92ef484b
Branch	codex/ui-code-quality
Original vulnerable tag	d347eb7c2404001fae0a2f43706be6e682afc7d4 - preserved
Runtime	Python 3.14.3, SQLite 3.51.3, macOS 26.6.2 ARM64
App packages	Flask 3.1.3; Werkzeug 3.1.9; Jinja2 3.1.6; Flask-WTF 1.3.0
Browser / accessibility	Playwright Chromium; axe-core 4.14.0
Tools	ZAP 2.17.0; Ruff 0.16.10; Node 24.19.0
Seed state	2 customers, 1 admin, 12 products, 4 categories, 6 sequential orders
Safety settings	Loopback bind only; debug/reload off; local assets; synthetic .invalid identities

Isolation. A clone of the exact source commit received a new virtualenv, database and signing key. The original server was stopped temporarily. Every smoke run was preceded by a stopped-server reset; another clean seed was used for the scan. The original server was restored after testing, and its database SHA-256 matched the original.

Method disclosure. Testing began with normal UI journeys, followed by known-weakness validation and ZAP. The assessor also built/reviewed the app and knew its intended weaknesses; this was not a blind assessment. Instructor stack approval and deadline were not independently verified.

Functional & regression results

Check	Result	Evidence
Existing unit tests	18 / 18 pass	regression/unittest.log
Additional isolated controls	16 / 16 pass	regression/focused-controls.log
Fresh clone install + seed + launch	Pass	regression/fresh-clone-summary.json
Normal complete demo journey	3 / 3 fresh-seed runs pass	smoke/clean-run-record.json
Ruff lint + format / pip check	Pass	regression/* logs
Syntax / templates / assets	14 Python files, 18 templates, 4 JS files; 2 checksums pass	regression/regression-summary.json

Customer demo path

Register a synthetic customer, sign in, search, open a product, add two items, verify the server total, remove and re-add, place a simulated order, verify own order/history, view profile, and sign out.

Administrator demo path

Use public /admin/login, create an unused product, edit its price/stock/name, delete it, verify the user listing and sign out. Customer access to management routes remains forbidden.

Controls checked

Role assignment and database role changes; hashed passwords and generic login errors; session/CSRF rotation and expiry; login throttling; invalid and oversize forms; cart ownership; server-calculated money and stock; historical price snapshots; atomic rollback after a forced checkout error; protected-route guards; loopback-only replay targets; deterministic seed and foreign-key integrity.

Replay observations

Observation	Fresh-seed result
customer_a -> /orders/4	HTTP 200; foreign order data returned
Search normal / true / false	1 / 12 / 0 results; boolean pair differs
Framing headers on home/catalogue	CSP absent; X-Frame-Options absent
Catalogue dependency	Local jquery-3.4.1.min.js referenced

One initial expiry-test harness configuration error was corrected and its failed log retained. The final expiry test used the real 3600-second configuration and a backdated token. No application failure or fix was attributed to that runner error. Python 3.11 itself was not executed; the actual runtime was 3.14.3.

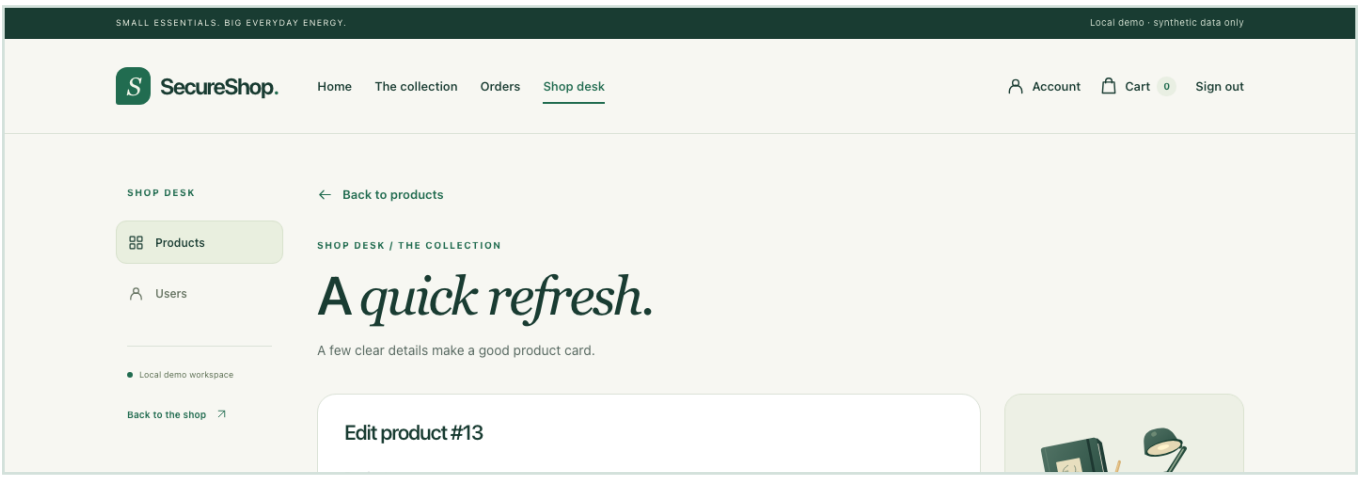
Browser, layout & accessibility

24 functional checks passed. They include live filtering across all four categories, AJAX add-to-cart and badge/fragment updates, password visibility, mobile navigation, keyboard actions, validation/empty/error pages, both role journeys and ordinary no-JavaScript form fallback.

Coverage	Result
14 route/role states x 5 widths	70 states pass at 320, 375, 768, 1024 and 1440px
Populated cart at the same widths	5 additional states pass; 75 total with no overflow
Navigation / form semantics	75 checks pass
Runtime / network	0 JS exceptions, unexpected failures or external requests
Axe targeted states	31 examined; 1 moderate best-practice violation
Contrast	11 incomplete rule instances; no conformance claim

Accessibility finding - admin product editor

Desktop /admin/products/new contains two unnamed complementary **aside** landmarks: .admin-sidebar and .editor-note-card. Axe reports **landmark-unique**, moderate severity. Distinct accessible names are a future improvement; no change was made. This is an accessibility finding, not an additional deliberate security weakness.



Shared editor capture (edit view). The naming finding was measured on product creation.

Axe reported no serious or critical violations in these states. Computed contrast spot checks had no failures and seven screenshot states were visually reviewed, but gradients, SVG text and all focus states need broader manual review. Four console logs were expected HTTP 400/403/404 responses from deliberate negative tests. They were not JavaScript exceptions.

ZAP scan record & coverage

Field	Actual setting/result
Date / time (UTC)	2026-10-08T07:08:49+00:00 to 2026-10-08T07:13:54+00:00
Target / proxy	http://127.0.0.1:8080 / 127.0.0.1:8090
ZAP / mode / context	2.17.0 / Protected / SecureShop baseline 2026-10-08
Effective include regex	^http://127[.]0[.]0[.]1:8080(?:/.*)?$
Session / form authentication	Cookie session; CSRF-aware /login; authenticated marker verified for each user
Completion	4 traditional spiders + 5 bounded active scans: 100%; passive queue 0
Requests / scope	529 active requests, all GET and all 127.0.0.1:8080; 0 off-origin discovered URLs
Excluded mutations	/logout, /checkout, admin product delete; forms not processed or posted

Phase	Role / target	Observed coverage or requests
Spider 0	Anonymous	40 URL rows
Spider 1	customer_a	55 rows; profile/history/orders authenticated
Spider 2	customer_b	58 rows; profile/history/orders authenticated
Spider 3	Admin	70 rows; users/products/new/edit authenticated
Active 0 + 1	Anonymous /search?q=notebook	131 + 164 requests; SQL detected on expanded rule
Active 2	customer_a /orders/1	66 requests
Active 3	customer_b /profile	89 requests
Active 4	Admin /admin/products	79 requests

Limits. Active scans use 14 selected built-in rules, recursion off. Only the general SQL rule was expanded from LOW to HIGH after the first pass. Time-based SQL, OS/code execution, XML expansion, cloud metadata and external-callback rules were excluded. There was no AJAX spider. POST mutations were covered by functional/regression/browser tests, not active-scanned. Crawl row counts include assets and are not unique application features.

Access control & injection

SS-01 - A01 Broken Access Control | Confirmed manually

What? A logged-in customer can read another customer's synthetic order by changing its sequential identifier.

Where? GET /orders/4 as customer_a; routes/customer.py order_detail.

Why? The handler requires login and selects the order by id without requiring user_id to match the session user. The history list still filters by owner.

Impact? Unauthorized disclosure of synthetic order ownership, products, prices and status. No order mutation or real-data impact was tested.

Evidence? replay/replay-20261008T070624147663Z.txt: HTTP 200, foreign_order_data_returned=true. Owner-filtered history and route guards pass regression checks. ZAP did not separately identify the ownership defect.

Planned later fix/retest. Add ownership to the order lookup; return the chosen denied response for non-owners. Repeat replay as both customers and keep each own order accessible. OWASP 2021 A01 -> 2025 A01.

SS-02 - A03 Injection | Confirmed manually and by ZAP

What? Raw search input changes the predicate of a read-only SQL SELECT.

Where? GET /search?q=; routes/public.py search; ZAP plugin 40018 (High, Medium confidence), alert 832.

Why? The deliberate query interpolates q rather than binding it. The handler constructs one SELECT and normal sqlite3 execute accepts a single statement; the connection is not read-only.

Impact? The observed comparisons change returned catalogue rows from 1 to 12 or 0. Write access, login bypass, operating-system access and arbitrary data extraction were not demonstrated.

Evidence? The fresh-seed replay returns normal=1, true=12, false=0. The expanded built-in SQL rule adds one confirmed ZAP alert. Raw scanner query values are omitted from the shareable export.

Planned later fix/retest. Bind the search value as a parameter. Ordinary search must still work, while the same comparison must no longer change SQL structure. OWASP 2021 A03 -> 2025 A05.

Framing & outdated component

SS-03 - A05 Security Misconfiguration | Confirmed

What / where? HTML responses omit framing protection: no CSP frame-ancestors and no X-Frame-Options. Source: app.py response hook.

Why / impact? The omission is intentional. The plain fixture at 127.0.0.1:9000 successfully embeds the catalogue. This proves framing is permitted; no deceptive clickjacking interaction was performed.

Evidence? Replay headers are absent on / and /products. ZAP records 43 anti-clickjacking and 53 missing-CSP instances. browser/framing-allowed.png records the allowed iframe.

Planned later fix/retest. Add CSP frame-ancestors 'none' and X-Frame-Options: DENY; verify framing is blocked. OWASP 2021 A05 -> 2025 A02.



Allowed local embedding; the fixture server was stopped after testing.

SS-04 - A06 Vulnerable/Outdated Component | Affected component confirmed

What / where? The catalogue loads local jquery-3.4.1.min.js from templates/public/catalogue.html. ZAP plugin 10003 reports Vulnerable JS Library.

Why / impact? jQuery 3.4.1 is affected by CVE-2020-11022; the patch begins at 3.5.0. Exploitation requires untrusted HTML reaching affected DOM methods. The app's use of text updates/filtering does not establish that prerequisite; browser exploitation is not claimed.

Evidence? Replay confirms the referenced path; vendor checksums pass. Live category filtering and AJAX cart operations work. The official download page identified 4.0.0 as latest on 08 Oct 2026; its local full build is stored but unused.

Planned later fix/retest. Upgrade only in the remediation phase, then repeat category filtering and AJAX/no-JS cart checks and the scanner. OWASP 2021 A06 -> 2025 A03 Software Supply Chain Failures.

Primary sources: jquery.com/download; jQuery security advisory GHSA-gxr4-xjj5-5px2.

Alert inventory & interpretation

Counting convention: 270 exported alert rows/IDs, grouped into 10 types by plugin ID and alert name. Repeated responses, URLs and phases create multiple instances. Informational detections are not vulnerabilities. SS-01 is a separate manual finding.

Alert type	Severity	Count	Assessment
Vulnerable JS library (10003)	Medium	1	confirmed
Missing anti-clickjacking header (10020)	Medium	43	confirmed
X-Content-Type-Options absent (10021)	Low	60	confirmed
Suspicious comment disclosure (10027)	Informational	1	false positive
Potential XSS attribute hotspot (10031)	Informational	9	unresolved
Server version disclosure (10036)	Low	71	confirmed
CSP header absent (10038)	Medium	53	confirmed
Authentication request identified (10111)	Informational	5	confirmed
Session response identified (10112)	Informational	26	confirmed
SQL Injection (40018)	High	1	confirmed

Severity	Types	Instances
High	1	1
Medium	3	97
Low	2	131
Informational	4	41

Additional observations

The server banner exposes Werkzeug/Python versions: a confirmed low-risk development-server observation. X-Content-Type-Options is absent under the intentionally omitted-header configuration. These are reported transparently alongside the four deliberate weaknesses; passing tests does not prove that only four possible defects exist.

Potential XSS / comment interpretation

The nine low-confidence HTML-attribute alerts are unresolved hotspots with benign contexts found on source review: fixed category choices and autoescaped quoted q values. No XSS execution was demonstrated. The suspicious-comment instance matches a normal jQuery implementation keyword, not sensitive application information, and is classified false positive for disclosure.

Assignment alignment & next gate

The complete eight-page faculty PDF was read. Its software requirements are distinct from the classroom notices on page 8. It supplies no verified deadline or final report template beyond Phase 3.

PDF requirement	Baseline evidence/status
Section 3 - approved local scope	Only owned loopback app and synthetic data; no off-origin scan URLs
Section 5 - public/customer/admin features	Fresh clone plus three complete demo runs, regression and browser coverage
Section 5 - instructor-approved stack	Flask stack matches an example; instructor approval remains unverified
Section 6 - four categories	A01, A03, A05 and A06 deliberately retained and independently observed
Section 7 - at least one old component	jQuery 3.4.1, latest 4.0.0, risk/advisory/prerequisites documented
Section 8 - Red Team method	UI-first journey documented; prior builder/code knowledge disclosed
Section 9 - ZAP scan record	Version, date, target, type, roles, counts, severity, URLs and evidence saved
Section 10 - investigation	What/Where/Why/Impact/Evidence for significant findings; alerts classified
Title/objectives - Fix -> Retest	Future authorized phase; not performed in this baseline task

Before remediation

Review the finding classifications and confirm any faculty-specific reporting format/stack approval. Preserve the tested source commit and evidence. The original vulnerable tag must continue to identify its original code, rather than being moved to the redesigned branch.

Future fix and retest plan

When separately authorized, make one change per controlled weakness, run regression/browser/replay again, repeat equivalent ZAP coverage, and collect before/after evidence. Upgrade jQuery without losing AJAX behavior. Recheck the release version on the remediation report date. No fixed tag or before/after success claim exists yet.

This report confirms the baseline testing performed. It does not establish complete penetration-test coverage, WCAG conformance, production readiness, or completion of the assignment's remediation/retest objectives.

Evidence & reproducible checks

All new evidence is under **evidence/test-suite-20261008T065601Z/**. The original baseline and UI-quality evidence remain unchanged. The companion Markdown report supplies individual links, full classifications and exact retest limits.

Artifact	Purpose
regression/regression-summary.json	34 tests, static checks, seed invariants, versions and runner history
regression/fresh-clone-summary.json	Fresh clone/venv/setup verification
smoke/clean-run-record.json + 3 logs	Three separately reset customer/admin journeys
replay/replay-20261008T070624147663Z.txt	Controlled weakness observations
browser/browser-results.json + summary	24 checks, 75 states, axe/contrast results and 23 screenshots
regression/zap-auth-verification.json	Actual authenticated profiles and admin-role checks
zap/scan-record.json + spider-coverage.json	Effective local scope, completed phases and role coverage
zap/alerts-sanitized.json + classifications	270 alert IDs, 10 types and investigation decisions
restoration.json	Original database checksum and server restoration
manifest-sha256.json	Checksums of the final evidence collection

Rerun normal checks in a fresh clone

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python seed.py
python app.py
```

From a second terminal: run `python tools/smoke.py` for normal journeys, or `python tools/replay.py` for the agreed small observation set. Stop the app before every seed reset. Existing regression tests run with `python -m unittest discover -v`. Optional quality tools are pinned in requirements-dev.txt.

The public scan export omits API keys, credentials, cookies, CSRF values and raw scanner payload/body fields. Exact traffic remains in the private Git-ignored ZAP session. Private scan sessions contain authentication material and are excluded from the shareable evidence package.

Sources checked: ZAP modes; ZAP authentication; ZAP anti-CSRF; official jQuery download and advisory linked on page 7. Assignment mapping is in assignment-requirements.md.